Examen Programación 1º
DAM/DAW Semipresencial
20 noviembre 2024
1ª Evaluación

I.E.S. Doctor Balmis

GENERALITAT VALENCIANA
Conselleria de Educación, Cultura y Deporte

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

Nombre	

Instrucciones para la realización de la prueba:

- Duración de la prueba **2 h 35 m** desde el comienzo de la misma.
- Lee atentamente el enunciado y no formules tus preguntas en voz alta, levanta la mano y espera a que el profesor te diga que tienes que hacer.
- Si te atascas en un apartado déjalo y pasa al siguiente. Cada apartado tiene su puntuación, independientemente del funcionamiento del programa final y el orden propuesto en el enunciado.
- Penalizará con **0,1 décimas** en cada uno de los siguientes casos:
  - Llaves mal puestas e instrucciones mal tabuladas dentro de un bloque.
  - No seguir las reglas de nomenclatura de C#.
  - Identificador de variable o método que a juicio del profesor no esté bien puesto porque no sigue las reglas descritas en classe.
- **No se puede usar ningún tipo de dispositivo USB, ni código de ejercicios o ejemplos realizados en clase. Solo los pdf de apuntes que se te proporcionarán para la ocasión**.
- En caso de pillar a algún alumno copiando o usando material no autorizado se le retirará el examen y su calificación en la evaluación será un suspenso.

Pregunta 1 (3 puntos)

La cadena "**Interstellar Food**" está abriendo franquicias por toda la galaxia y los platos variarán de precio dependiendo de la escasez de los ingredientes en el planeta en el que esté el restaurante y de las especies que lo habiten. Para ello, necesita un programa que le permita calcular los precios de los platos de su menú dependiendo del planeta en la que estemos, la especie del comensal y obviamente el plato elegido.

El restaurante ofrece tres platos principales:

- **Nebulosa de Nutrientes**: Una nube de gas espacial rica en minerales.
  Tiene el código 1 en el menú y cuesta 15 créditos intergalácticos en todos los planetas.
- **Estrella Enana a la Parrilla**: Ideal para paladares que disfrutan de sabores intensos.
  Tiene el código 2 en el menú y cuesta 40 créditos intergalácticos en los planetas **Ganimedes** y **Raticulin** y 30 créditos intergalácticos en el resto.
- **Agujero Negro con Salsa de Singularidad**: ¡No apto para estómagos sensibles!
  Tiene el código 3 en el menú y cuesta 45 créditos intergalácticos en el planeta **Pandora**, 50 en el planeta **Acheron** y 60 en el resto.

Descuentos a aplicar:

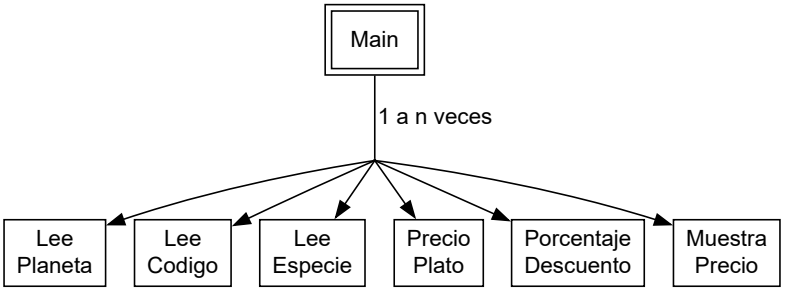
- **Gleepglorp**: Obtienen un 10% de descuento en cualquier plato.
- **Floofle**: Solo pagan la mitad si piden la Estrella Enana a la Parrilla.
- **Zorp**: Comen gratis, ¡son los críticos gastronómicos más temidos de la galaxia!
- Cualquier otra especie que no sea una de las anteriores no tendrá descuento.

El programa debe:

1. Seguir el diagrama de módulo de la imagen y usar switch de expresión para las estructuras condicionales.

2. Pedir al usuario que introduzca el código del plato (un número del 1 al 3) y el nombre de la especie.

   > 📌 **Nota**: No hace falta que muestres un menú con los platos. Solo debe introducir un código y si el número es incorrecto deberá mostrar un aviso y pedirlo de nuevo.



3. Calcular el precio final teniendo en cuenta el plato y la especie.
4. Mostrar el precio final por consola.
5. Repetir los pasos 1-3 hasta que se introduzca la especie "Humano", momento en el cual se mostrará el mensaje *'¡Humanos no permitidos!, Avisando a la policía del sistema'* y finalizará el programa.

Ejemplo de ejecución:

```
Introduce el planeta: Acheron
Introduce el código del plato (1-3): 5
Código de plato no válido. Inténtalo de nuevo.
Introduce el código del plato (1-3): 3
Introduce la especie: Protoss
Precio base: 50
Total a pagar: 50 créditos intergalácticos.

Introduce el planeta: Kune
Introduce el código del plato (1-3): 2
Introduce la especie: Gleepglorp
Precio base: 30
Descuento: 10%
Total a pagar: 27 créditos intergalácticos.

Introduce el planeta: Raticulin
Introduce el código del plato (1-3): 3
Introduce la especie: Humano
¡Humanos no permitidos!, Avisando a la policía del sistema...
```

Criterios de calificación:

- (✅ **0,5 pts**) Define correctamente el módulo que lee código del plato.
- (✅ **1 pts**) Define correctamente el módulo que devuelve el precio usando switch de expresión.
- (✅ **0,5 pts**) Define correctamente el módulo que devuelve el porcentaje de descuento usando switch de expresión.
- (✅ **0,5 pts**) Define correctamente el módulo que muestra el precio final o el aviso de humanos no permitidos.
- (✅ **0,5 pts**) El programa compila y ejecuta correctamente de acuerdo a la especificación.

Pregunta 2 (3 puntos)

Tenemos que desarrollar un programa que descifre un mensaje secreto codificado en una secuencia numérica.

Para ello, se te proporciona una **secuencia de números enteros**. Cada número de la secuencia representa una letra del alfabeto español (incluyendo la Ñ). Para descifrar el mensaje, debes calcular el resto de la división de cada número entre la longitud del alfabeto (**incluyendo el carácter espacio en blanco**). El resultado de esta operación te dará la posición de la letra correspondiente en el alfabeto.

El alfabeto a utilizar lo definiremos en la cadena: ***" ABCDEFGHIJKLMNÑOPQRSTUVWXYZ"*** (observa que incluye un espacio en la posición de índice 0).

Por ejemplo:

Si la longitud de la cadena del alfabeto 28 y un número en la secuencia es 31, el resto de la división (5 % 28) es 5. Esto significa que la letra correspondiente es la que está en el índice 5 de la cadena con el alfabeto, que en este caso es la letra 'E'.

Ejemplo de ejecución:

```
Secuencia recibida: 5 38 61 103 115 149 171 205 240 252 283 324 355 383 397 423 469 492
Mensaje Secreto: EJERCICIO CORRECTO
```

Debes implementar los siguientes métodos:

`int[] SecuenciaNumerica(string entrada)` : Esta función recibirá una cadena de texto con la secuencia numérica separada por espacios. Deberá convertir esta cadena en un array de enteros.
`string Descifrar(int[] secuencia)` : Esta función recibirá el array de enteros generado por la función SecuenciaNumerica. Deberá descifrar el mensaje utilizando el método del resto de la división explicado anteriormente. Para construir el mensaje descifrado, debes utilizar un `StringBuilder` .

Criterios de calificación:

- (✅ **1,25 pts**) Define correctamente el método `SecuenciaNumerica` .
- (✅ **1,25 pts**) Define correctamente el método `Descifrar` utilizando un `StringBuilder` para construir el mensaje, penalizará **0,5 pts** si no se usa `StringBuilder` para su realización.
- (✅ **0,5 pts**) El programa compila y ejecuta correctamente de acuerdo a la especificación.

Pregunta 3 (4 puntos)

Eres un bardo moderno que, en lugar de laúd, utiliza un programa en C# para componer sus poemas. Tu musa es un tanto peculiar y te exige que los poemas tengan una estructura muy específica que especificaremos a continuación.

El programa debe solicitar al usuario dos datos:

- **Número de estrofas**: Un número entero que indica la cantidad de estrofas del poema.
- **Frase base**: Una frase que servirá como base para la construcción del poema.

Para ello definiremos un método `ObtenerEntradaUsuario`: Se encarga de obtener del usuario el número de estrofas y la frase base y devolverlos en forma de tupla.

Con estos datos, el programa generará un poema con la siguiente estructura:

1. Cada **estrofa** contiene tantos versos como número de estrofas hay en poema.
2. Cada **verso** está compuesto por la **frase base repetida** un número determinado de veces, empezando en 1 e incrementándose en uno en cada repetición. Por lo que ek último verso de cada estrofa tendrá el mismo número de repeticiones que el número de versos en la estrofa.
3. Cada repetición de la frase base en un verso debe estar "***realzada***", esto es:
    - La primera letra de cada palabra debe pasarse a mayúscula.
    - Se añadirán signos de admiración al inicio y al final, a partir de la segunda repetición de la frase base en el verso. Empezando en una admiración e incrementándose el número de admiraciones en cada repetición.

Ejemplo de ejecución:

```
Número de estrofas: 3
Frase base: oh musa

Oh Musa
Oh Musa, ¡Oh Musa!
Oh Musa, ¡Oh Musa!, ¡¡Oh Musa!!

Oh Musa
Oh Musa, ¡Oh Musa!
Oh Musa, ¡Oh Musa!, ¡¡Oh Musa!!

Oh Musa
Oh Musa, ¡Oh Musa!
Oh Musa, ¡Oh Musa!, ¡¡Oh Musa!!
```

Fíjate que el poema está formado por 3 estrofas, cada una tiene 3 versos y cada verso contiene la frase base repetida y realzada según las reglas descritas y donde en el último verso tenemos el mismo número de repeticiones de la frase base que el número de versos en la estrofa.

Para este ejercicio, debes dividir tu programa en los siguientes métodos con las siguientes restricciones de diseño:

1. Método `Realza` : Recibe la **frase base** y un número de admiraciones a añadir, y devuelve la frase "realzada" según las reglas descritas.
   > 📌 **Nota**: Se valorará el uso de `StringBuilder` para construir la frase realzada.
2. Método `GeneraVerso` : Recibe la **frase base** y un número de repeticiones, y devuelve un verso con la frase repetida y realzada según el número de repeticiones separadas por comas como en el ejemplo.
   > 📌 **Nota**: Se valorará el uso del método estático `String.Join` para construir el verso.
3. Método `GenerarPoema` que recibirá la entrada y devolverá el poema completo generando estrofas con un **bucle for anidado**. Fíjate que entre cada estrofa hay un salto de línea pero después de la última no.

Criterios de calificación:

- (✅ **0,5 pts**) Método `ObtenerEntradaUsuario` siguiendo las especificaciones.
- (✅ **1 pts**) Método `Realza` siguiendo las especificaciones, penalizará **0,5 pts** si no se usa `StringBuilder` para su realización.
- (✅ **1 pts**) Método `GeneraVerso` siguiendo las especificaciones, penalizará **0,5 pts** si no se usa `String.Join` para su realización.
- (✅ **1 pts**) Método `GenerarPoema` siguiendo las especificaciones.
- (✅ **0,5 pts**) El programa compila y ejecuta correctamente de acuerdo a la especificación.